

Networked Music Performance in Max: from Tool to Prototype

ADCx Copenhagen **26**

April 28th, 2026

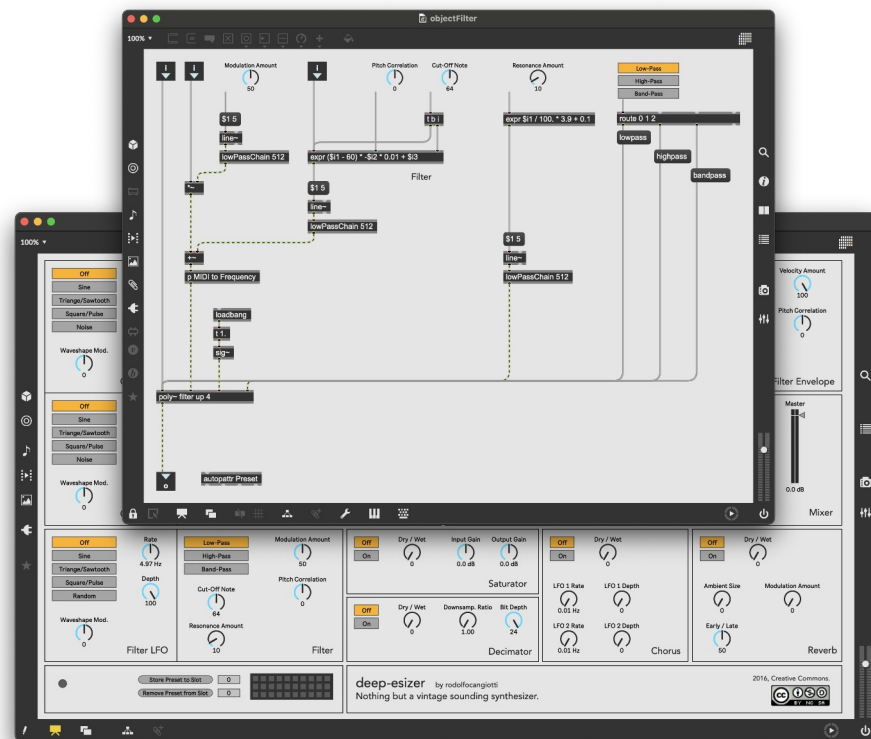


**AALBORG
UNIVERSITY**

Cycling '74 Max

What Is It?

- Visual programming language development environment
- Authored by Miller Puckette
- Acquired by David Zicarelli and currently sold by Cycling '74



Networked Music Performance – p. 1

Definition:

- Musical performative act using devices connected via a network

NMP Aesthetics:

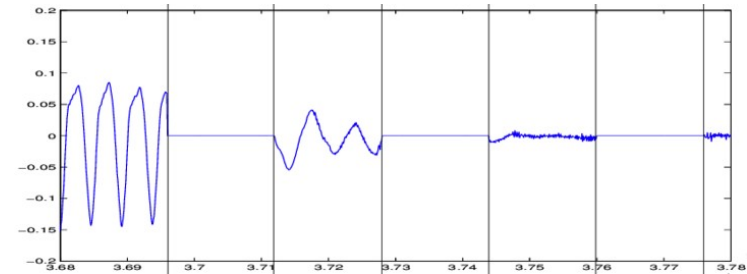
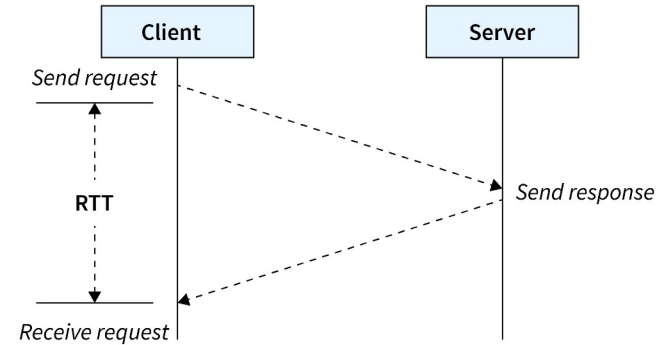
- Alternative to in-person performance, when this latter is not possible
- Space-time limitation reduction



Networked Music Performance – p. 2

NMP Issues:

- Network latency
 - ↳ up to 20-30 ms for audio stream
 - ↳ up to 150 ms for video stream
- Data loss
 - ↳ less is better



Networked Music Performance – p. 3

NMP Taxonomy:

- Network architecture
 - ↳ client-server, P2P
- Network area
 - ↳ LAN, WAN
- Network protocol
 - ↳ TCP, UDP, RTP
- Transmitted data
 - ↳ audio, video, features
- Synchronization algorithm
 - ↳ no sync, clock sync
- Data compression
 - ↳ uncompressed, lossless, lossy

NMP in Max: State of the Art – p. 1

List of Native Objects:

- `udpsend` and `udpreceive`
 - ↳  UDP protocol
 - ↳  only for control data
- `jit.net.send` and `jit.net.receive`
 - ↳  TCP protocol
 - ↳  only for video streams
- `mxj`
 - ↳  Java classes for data exchange via TCP/UDP
 - ↳  JDK on host machine

NMP in Max: State of the Art – p. 2

List of Externals or Projects:

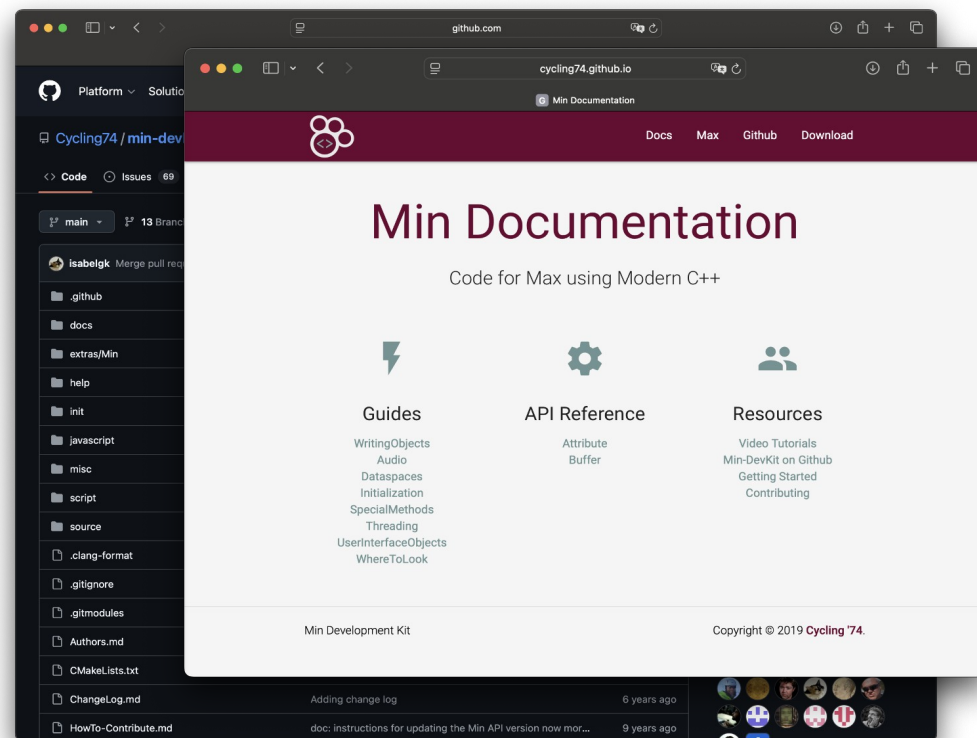
- `odot` library
 - ↳  OSC protocol on top of TCP/UDP
 - ↳  only for control data
- `Quaxtrip` library
 - ↳  JackTrip wrapper
 - ↳  built upon Pure Data
- `Conductor` project
 - ↳  up to five performers plus a conductor
 - ↳  multiple dependencies

Is it possible to produce an external,
fully-featured Max object for
Networked Music Performance, with
no dependencies?

Cycling '74 Min DevKit – p. 1

Key Points:

- High level declarative API
- Written in modern C++
- Extensively documented
- MIT license of use



Cycling '74 Min DevKit – p. 2

DevKit Essentials (pt. 1):

- Base classes:
 - `sample_operator`, for sample based DSP
 - `vector_operator`, for vector based DSP
- `operator()`, for the actual DSP computation
- `inlet` and `outlet` class attributes, for object I/O
- `argument<T>` class attributes, for object arguments
- `attribute<T>` class attributes, for object (named) attributes

Cycling '74 Min DevKit – p. 3

DevKit Essentials (pt. 2):

- `message` class attributes, to define object's behavior when receiving the message in question
- Magic messages, called by Max at runtime (e.g. `setup`, `dspsetup`, etc.)
- Macros, for class metadata or to declare lambda functions called by `argument`, `attribute` or `message`

Max Object for NMP – p. 1

Context:

- This research is part of a NMP project.

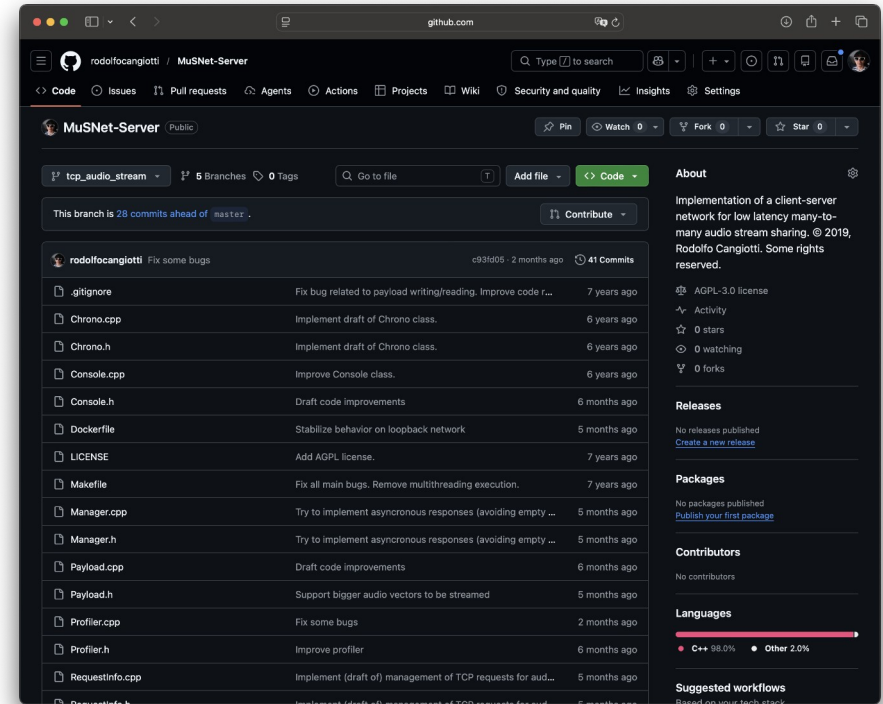
Project Pillars (pt. 1):

- Ecosystem for low-latency NMPs
 - ↳ cross platform compliance
 - ↳ multiple target builds
 - ↳ streamlined configuration and usage

Max Object for NMP – p. 2

Project Pillars (pt. 2):

- Client-server architecture
- *Ad hoc* defined data transmission protocol



Max Object for NMP – p. 3

Object Requirements:

- Stereo audio I/O
- Control messages
 - ↳ Server IP (and port)
 - ↳ Audio exchange mode
 - ↳ Connection and disconnection
 - ↳ Session state

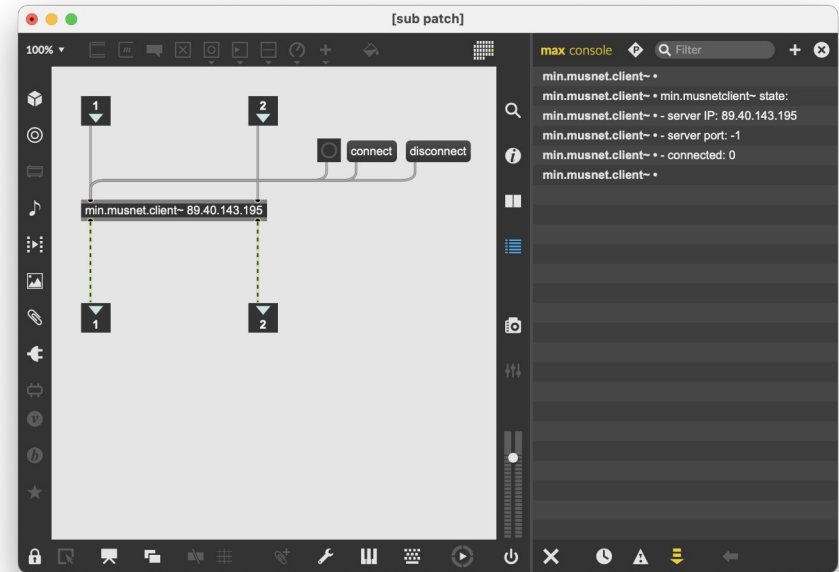
Implementation:

- Reuse of project codebase, via related APIs

Max Object for NMP – p. 4

Outcome:

- Stable beta version of the object
 - ↳ ● TCP/UDP protocol
 - ↳ ● for audio streams
 - ↳ ● no dependencies



Thank you for your attention!

Further information at:

